Phase 3 Report

Support Ticket Management System

SRS-based Implementation Verification

Document Version: 1.0

Date: 2026-07-18

Requirements Source: docs/SRS.pdf

Implementation Source: Local project source code

Prepared as part of the academic project documentation for the Ticket Management System.

Generated on 2026-07-18 from verified local implementation.

1. Introduction

This report documents the implementation status of the Support Ticket Management System relative to the Software Requirements Specification (SRS). The source code serves as the implementation truth and the SRS serves as the requirements truth. No unimplemented feature has been reported as complete, and all test results are based on actual local execution.

Supplementary outputs in docs/phase-3/ include:

- backend-test-results.md (77 individual test records)
- C4 architecture diagrams (6 diagrams in SVG and Mermaid formats)
- UI screenshots (17 captures including before/after interactions)
- CI configuration report (.github/workflows/ci.yml analysis)
- Terminal evidence files (raw test output)

2. Implemented Requirements

Based on mapping SRS (pages 11-12 and related sections) to source code:

ID	Title	Status	Source Code Reference
FR-01	Create support ticket	Implemented	tickets/services/ticket_service.py
FR-03	Ticket status management	Implemented	tickets/services/ticket_service.py
FR-04	User-agent conversation	Implemented	tickets/services/message_service.py
FR-06	Customer ticket tracking	Implemented	tickets/services/ticket_service.py
FR-10	Username/password auth	Implemented	accounts/session_views.py
FR-11	Role-based access control	Implemented	accounts/permissions.py
FR-12	User & role management	Implemented	accounts/views.py
FR-13	Ticket filter & search	Implemented	tickets/views.py
FR-14	Swagger/OpenAPI docs	Implemented	ticketProject/urls.py
NFR-05	Browser-based access	Implemented	React SPA / Browser

Key implementation evidence: Ticket creation in TicketService.create_ticket and frontend CreateTicket.tsx. Status transitions (OPEN -> IN_PROGRESS -> RESOLVED -> CLOSED) with enforced valid transitions. Role-specific dashboards for customer/agent/admin. Session-based authentication with CSRF protection. Full details in requirements-traceability.md.

3. Missing or Partial Requirements

The following requirements from SRS are not fully implemented:

ID	Title	Status	Notes
FR-02	Auto ticket assignment	Not Implemented	Only manual admin assignment exists
FR-05	Full admin dashboard	Partial	Counts exist; response time/ratings missing
FR-07	Smart classification	Partial	Manual priority/category; no auto-classification
FR-08	FAQ auto-reply	Not Implemented	No FAQ/knowledge-base module
FR-09	Agent evaluation	Not Implemented	Workload counts exist; no rating/SLA
NFR-01	100 concurrent users	Not Verified	No load-test evidence in repo
NFR-02	Security (HTTPS)	Partial	Hashing/CSRF exist; HTTPS not configured locally
NFR-03	Reliability/backup	Partial	Structured errors exist; no backup automation
NFR-04	Integration readiness	Partial	OpenAPI/docs exist; no external integrations

Aligned with docs/limits-and-roadmap.md: missing email notifications, file attachments, real-time updates, and automated frontend tests.

4. Updated C4 Diagrams

The existing C4 model in docs/c4-model.md has been verified against the current implementation and exported as SVG diagrams in docs/phase-3/c4/:

- 01-system-context.svg: System Context - three user roles interacting with the system
- 02-container.svg: Container - React SPA, Django REST API, SQLite, Swagger/ReDoc
- 03-backend-components.svg: Backend Components - accounts, tickets, dashboard apps with services
- 04-frontend-components.svg: Frontend Components - pages, hooks, API layer, shared components
- 05-login-flow.svg: Login Flow - CSRF token fetch, session check, authenticate, redirect
- 06-ticket-lifecycle.svg: Ticket Lifecycle - create, assign, message, status transitions

Containers: React SPA (TypeScript/Vite, port 5173), Django REST API (Python/DRF, port 8000), SQLite database (file-based), Swagger UI/ReDoc (drf-spectacular). Backend architecture follows View -> Service -> Model layering. Mermaid source files (.mmd) are included alongside SVGs.

5. Backend Test Results

Local execution matching the CI command:

```
python manage.py test accounts.tests tickets.tests dashboard.tests
Found 77 test(s)
Ran 77 tests in 98.101s
OK
```

Summary Statistics:

Total Tests	**77**
Passed	77
Failed	0
Errors	0
Pass Rate	100%

By Module:

- tickets.tests: 40 tests (ticket CRUD, messaging, categories, status transitions)
- dashboard.tests: 14 tests (role-specific overviews, agent workload)

WARNING entries in test logs correspond to expected negative tests (401/403/400 responses) and do not indicate failures. Full per-test details in backend-test-results.md. Raw terminal output in evidence/backend-test-run.txt.

6. Main UI Screenshots

The following screenshots were captured from the running application at http://127.0.0.1:5173:

01-login.png - Login page
02-customer-dashboard.png - Customer dashboard view
03-ticket-list.png - Ticket list with filtering options
04-ticket-details.png - Ticket details with messages
05-ticket-conversation.png - Ticket conversation thread
06-admin-dashboard.png - Admin management dashboard
07-agent-dashboard.png - Agent dashboard view
08-ticket-assignment.png - Ticket assignment UI
09-swagger.png - Swagger API documentation page

7. Frontend Interaction (Before/After)

Before/after screenshot pairs demonstrating UI interactions:

- Validation - empty form submit: before-after-validation-empty-submit.png
- Validation - invalid login credentials: before-after-validation-invalid-login.png
- Ticket creation - before and after: before-after-ticket-creation-before/after.png
- Status update - before and after: before-after-status-update-before/after.png
- List filtering - before and after: before-after-filtering-before/after.png

Screenshots captured from real UI execution. No test credentials are published in this report.

8. CI Configuration

File: .github/workflows/ci.yml - Defines two pipeline jobs:

Backend Pipeline (Python 3.12):

1. actions/checkout@v4 - Clone repository
2. actions/setup-python@v5 with Python 3.12
3. pip install -r requirements.txt
4. python manage.py makemigrations --check --dry-run
5. python manage.py migrate
6. python manage.py check
7. python manage.py test accounts.tests tickets.tests dashboard.tests

Frontend Pipeline (Node 20):

1. actions/checkout@v4 - Clone repository
2. actions/setup-node@v4, Node 20, npm cache on package-lock.json
3. npm ci - Clean lockfile-faithful install
4. npx tsc --noEmit - TypeScript type checking
5. npm run build - Production bundle (tsc + vite build)

NOTICE: No successful remote GitHub Actions run is claimed. The workflow configuration is documented as-is. Backend tests matching the CI command were run and verified locally. Frontend tsc --noEmit and npm run build were not re-executed for this Phase 3 evidence pack unless separately recorded.

CI does not include: deployment step, frontend unit/E2E tests, Docker build,

coverage upload, artifact publishing, or scheduled runs.

9. Limitations

1. SQLite database (suitable for development, not production-scale)
2. No email notifications, file attachments, or WebSocket real-time updates
3. No FAQ auto-reply system or user satisfaction ratings
4. No automatic ticket assignment - only manual admin assignment exists
5. No automated Frontend/E2E tests
6. Session/cookie-based authentication (no JWT)
7. 100-user concurrent load scenario not verified (no load-test evidence)
8. HTTP only in local development (no HTTPS configured)
9. No response-time tracking or SLA monitoring implemented
10. No data backup/recovery automation

10. Conclusion

The core operational functionality of the Ticket Management System has been implemented according to the initial SRS requirements for ticket creation, tracking, conversation, roles, and basic dashboards. This is verified by 77 backend tests running at a 100% pass rate.

The main gap relative to SRS and Vision documents is in intelligent and analytical capabilities: automatic assignment, FAQ auto-reply, satisfaction measurement, and response-time tracking. These features are documented in Vision as second-priority or future-release items.

C4 architecture diagrams have been updated to match the current implementation. Requirements traceability provides a complete mapping from each SRS requirement to source code, tests, and screenshots. All test results and screenshots are from actual execution with no fabricated data.

All Phase 3 documentation is complete and available in docs/phase-3/.

End of Phase 3 Report. Generated on 2026-07-18 from local project source code.